

```

#include <iostream>
#include <algorithm> // For std::sort

using namespace std;

template<class T>
class LinkedList
{
public:
    class Node
    {
    public:
        Node() : next(nullptr), data(0) {}
        Node(T d) : next(nullptr), data(d) {}
        Node(Node *n) : next(n), data(0) {}
        Node(Node *n, T d) : next(n), data(d) {}
        T data;
        Node *next;
    };
    LinkedList() : head(nullptr) {}
    ~LinkedList() {
        Node *current = head;
        while (current) {
            Node *next = current->next;
            delete current;
            current = next;
        }
        head = nullptr;
    }
    bool addFromHead(T d) {
        Node *newNode = new Node(d);
        if (!newNode) return false;
        newNode->next = head;
        head = newNode;
        return true;
    }
    bool addFromTail(T d) {
        Node *newNode = new Node(d);
        if (!newNode) return false;
        if (!head) {
            head = newNode;
        } else {
            Node *current = head;
            while (current->next) {
                current = current->next;
            }
            current->next = newNode;
        }
        return true;
    }
    bool removeFromHead() {
        if (!head) return false;
        Node *temp = head;
        head = head->next;
        delete temp;
        return true;
    }
    bool removeFromTail() {
        if (!head) return false;
        if (!head->next) {

```

```

        delete head;
        head = nullptr;
    } else {
        Node *current = head;
        while (current->next->next) {
            current = current->next;
        }
        delete current->next;
        current->next = nullptr;
    }
    return true;
}

friend ostream &operator<<(ostream &o, LinkedList *l) {
    o << "{";
    Node *current = l->head;
    while (current) {
        o << current->data;
        current = current->next;
        if (current) o << ", ";
    }
    o << "}";
    return o;
}

size_t getSize() const {
    size_t count = 0;
    Node *current = head;
    while (current) {
        count++;
        current = current->next;
    }
    return count;
}

bool searchData(T d) {
    Node *current = head;
    while (current) {
        if (current->data == d) return true;
        current = current->next;
    }
    return false;
}

T operator[](int i) {
    int count = 0;
    Node *current = head;
    while (current) {
        if (count == i) return current->data;
        count++;
        current = current->next;
    }
    throw out_of_range("Index out of range");
}

```

protected:

```

    Node *head;
};

```

```

template<class T>
class Set : private LinkedList<T>
{
public:
    Set<T>() : LinkedList<T>() {}
}

```

```

bool addElement(T d) {
    if (this->searchData(d)) return false;
    return this->addFromTail(d);
}

```

```

friend ostream &operator<<(ostream &o, Set<T> *s) {
    o << "{";
    T *arr = s->toArray();
    int size = s->getSize();
    if (size > 0) {
        o << arr[0];
        for (int i = 1; i < size; ++i) {
            o << ", " << arr[i];
        }
    }
    o << "}\n";
    delete[] arr;
    return o;
}

```

```

Set<T>* operator+(Set<T> *s) {
    Set<T> *result = new Set<T>();
    T *arr1 = this->toArray();
    T *arr2 = s->toArray();
    int size1 = this->getSize();
    int size2 = s->getSize();

    for (int i = 0; i < size1; ++i) {
        result->addElement(arr1[i]);
    }
    for (int i = 0; i < size2; ++i) {
        result->addElement(arr2[i]);
    }
    delete[] arr1;
    delete[] arr2;
    return result;
}

```

```

Set<T>* operator-(Set<T> *s) {
    Set<T> *result = new Set<T>();
    T *arr1 = this->toArray();
    T *arr2 = s->toArray();
    int size1 = this->getSize();
    int size2 = s->getSize();

    for (int i = 0; i < size1; ++i) {
        if (!s->searchData(arr1[i])) {
            result->addElement(arr1[i]);
        }
    }
    delete[] arr1;
    delete[] arr2;
    return result;
}

```

```

Set<T>* operator*(Set<T> *s) {
    Set<T> *result = new Set<T>();
    T *arr1 = this->toArray();
    int size1 = this->getSize();

```

```

    for (int i = 0; i < size1; ++i) {
        if (s->searchData(arr1[i])) {
            result->addElement(arr1[i]);
        }
    }
    delete[] arr1;
    return result;
}

```

```

private:
    T* toArray() {
        int size = this->getSize();
        T *arr = new T[size];
        typename LinkedList<T>::Node *current = this->head;
        for (int i = 0; i < size; ++i) {
            arr[i] = current->data;
            current = current->next;
        }
        return arr;
    }
};

```